

Reviewer Pack — Minimal Order of Grothendieck Groups in Boolean Functions vs...

Entry 39ffcdc3bc28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 22:49:23 UTC

Minimal Order of Grothendieck Groups in Boolean Functions vs. Circuit Monotone Width

Entry ID: 39ffcdc3bc28 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 22:49:23 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Grothendieck Groups) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every n -input boolean function f , the minimal order of its associated Grothendieck group G_f is linearly correlated with its circuit monotone width $w(f)$, such that $\log(|G_f|) = \Theta(w(f))$.

Rationale (proposer’s reasoning):

Grothendieck groups provide a categorical framework for studying algebraic structures, and their orders could potentially reveal hidden complexity in boolean functions. A connection between Grothendieck group order and circuit monotone width might expose new insights into the structure of boolean functions.

Taxonomy category: GrothendieckGroupMonotoneWidth (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f3786cad4db1a906

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 95% of generated n -input boolean functions f show that $\log(|G_f|) = \Theta(w(f))$, with an absolute error in the correlation coefficient ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Grothendieck Groups" AND "Boolean Circuit Complexity" - "Minimal order of Grothendieck groups" AND "circuit monotone width" - "Algebraic Geometry" AND "linear correlation" WITHIN 1000 OF "Boolean functions"

Top relevant hits considered: - [http://arxiv.org/abs/2112.12010v1] Algebraic geometry in mixed characteristic - [http://arxiv.org/abs/2505.02264v1] A note on gluing: a pillar of algebraic geometry - [http://arxiv.org/abs/2002.06154v2] Epsilon local rigidity and numerical algebraic geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def circuit_monotone_width(f):
        n = int(math.log2(len(f)))
        max_ones = 0
        for i in range(n):
            ones = sum(f[j] for j in range(i*2**(n-1), (i+1)*2**(n-1)))
            if ones > max_ones:
                max_ones = ones
        return max_ones

    def grothendieck_group_order(f):
        n = int(math.log2(len(f)))
        G = [0] * (2**n)
        for i in range(2**n):
            if f[i] == 1:
                G[i] += 1
        return max(G) + 1

    def correlation(xs, ys):
        n = len(xs)
        mean_x = sum(xs) / n
        mean_y = sum(ys) / n
        cov = sum((xs[i] - mean_x) * (ys[i] - mean_y) for i in range(n)) / n
```

```

var_x = sum((xs[i] - mean_x)**2 for i in range(n)) / n
var_y = sum((ys[i] - mean_y)**2 for i in range(n)) / n
return cov / (math.sqrt(var_x) * math.sqrt(var_y))

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    f = generate_boolean_function(n)
    w_f = circuit_monotone_width(f)
    log_G_f = math.log(grothendieck_group_order(f))
    results.append((w_f, log_G_f))

if len(results) < 30:
    return {
        "metric_name": "log_grothendieck_group_order",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

ws, log_G_fs = zip(*results)
correlation_coefficient = correlation(ws, log_G_fs)
if abs(correlation_coefficient) < 0.95:
    return {
        "metric_name": "log_grothendieck_group_order",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": f"correlation_coefficient={correlation_coefficient}"
    }

return {
    "metric_name": "log_grothendieck_group_order",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_v
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.95:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    for r in results:
        if not r["conjecture_holds"]:
            counterexample = r["counterexample"]
            break
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[result.index]}\n")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffbdc2b6.py", line 94, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffbdc2b6.py", line 54, in run_trial
    w_f = circuit_monotone_width(f)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffbdc2b6.py", line 28, in circuit_monotone_width
    ones = sum(f[j] for j in range(i*2**(n-1), (i+1)*2**(n-1)))
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ffbdc2b6.py", line 28, in <genexpr>
    ones = sum(f[j] for j in range(i*2**(n-1), (i+1)*2**(n-1)))
           ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	34539
2	propose	ollama_remote	glm4:latest	0	0	24275
3	preregistration	ollama_remote	glm4:latest	0	0	10926
4	novelty	ollama_remote	glm4:latest	0	0	17441
5	novelty	ollama_remote	glm4:latest	0	0	12365
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16691
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42183
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11563
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23174
10	judge	ollama_remote	glm4:latest	0	0	16390

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 209547 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/39ffcdc3bc28.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/39ffcdc3bc28.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/39ffcdc3bc28.tar.gz* (if generated)